



JN

Mon Parcours

Développeur Web & IA

Phase 1 — Semaines 1 à 8

par Jopie Nlemvo

8

Semaines

3

Langages

5

Projets

1

Site Live

Introduction	3
Semaine 1 — Internet & Environnement	4
Semaine 2 — HTML	5
Semaine 3 — CSS	6
Semaine 4 — JavaScript	7
Semaine 5 — DOM & To-Do List	8
Semaine 6 — Git & GitHub	9
Semaine 7 — Prompt Engineering	10
Semaine 8 — Python	11
Tableau des Codes Essentiels	12
Ma Procédure en 10 Étapes	14
Mes Accomplissements	15

Développeur Web & IA

Phase 1 — Semaines 1 à 8

par Jopie Nlemvo

8

Semaines

3

Langages

5

Projets

1

Site Live

Introduction

Ce livre numérique retrace mon parcours complet de formation en développement web et intelligence artificielle. En 8 semaines, je suis passé de zéro à la création d'un vrai site web professionnel en ligne, accessible dans le monde entier.

Mon Profil	Jopie Nlemvo
GitHub	nlemvojopie-star
Site en ligne	nlemvojopie-star.github.io/formation-web
Email	nlemvojopie@gmail.com
Durée de formation	8 semaines — Phase 1
Objectif	Développeur Web & IA dans une organisation innovante

Semaine 1 ■ Internet & Environnement

Objectif : Comprendre le fonctionnement d'Internet et installer les outils de développement.

Concepts clés appris

✓ Internet : communication entre client et serveur via HTTP/HTTPS

✓ Frontend (HTML/CSS/JS) vs Backend (Python/Node.js/SQL)

✓ L'IA Générative : LLMs comme Claude prédisent le texte

✓ VSCode : éditeur de code professionnel gratuit

✓ Node.js v22.22.2 et npm 10.9.7 installés

✓ Terminal : interface pour communiquer avec l'ordinateur

Exercice réalisé

Explorer Claude.ai avec des prompts, inspecter une page web avec F12, installer VSCode et ses extensions (Prettier, Live Server, French Language Pack), utiliser le terminal.

Code essentiel

```
node --version → v22.22.2
npm --version → 10.9.7
git --version → 2.54.0
```

■ **Accomplissement : Environnement de développement 100% opérationnel sur Windows 11**

Semaine 2 ■■ HTML

Objectif : Créer la structure d'une page web avec HTML.

Concepts clés appris

✓ HTML = HyperText Markup Language : squelette de la page

✓ Structure de base : DOCTYPE, html, head, body

✓ Balises de titres : h1 à h6

✓ Balises de contenu : p, ul, ol, li

✓ Liens et images : a href, img src, alt

✓ Balises sémantiques : header, main, section, footer

Exercice réalisé

Créer une carte de visite personnelle avec nom, description, liste de 5 passions et email cliquable. Publier avec Live Server sur 127.0.0.1:5500.

Code essentiel

```
Ma carte de visite
```

```
Bonjour, je m'appelle Jopie !  
Développeur Web en formation
```

```
L'intelligence artificielle
```

■ **Accomplissement :** Première page web créée et visible dans le navigateur

Semaine 3 ■ CSS

Objectif : Styliser et embellir la page web avec CSS.

Concepts clés appris

✓ CSS = Cascading Style Sheets : apparence de la page

✓ Sélecteurs : element, .class, #id

✓ Propriétés : color, background-color, font-family

✓ Box model : margin, padding, border

✓ Google Fonts : polices externes avec @import

✓ Transitions et effets :hover pour les animations

Exercice réalisé

Styliser la carte de visite : fond coloré, police Poppins, header sombre, sous-titres bleus avec bordure, liens rouges avec effet hover, animations sur les listes.

Code essentiel

```
@import url("https://fonts.googleapis.com/css2?family=Poppins");

body { background-color: #f0f4f8; font-family: Poppins; }
h1 { background: #2c3e50; color: white; padding: 40px; }
h2 { color: #2980b9; border-bottom: 2px solid #2980b9; }
li:hover { padding-left: 12px; transition: 0.3s; }
```

■ **Accomplissement :** Page web avec design professionnel, animations et police Google Fonts

Semaine 4 ■ JavaScript

Objectif : Rendre la page web interactive avec JavaScript.

Concepts clés appris

- ✓ JS = langage qui donne vie aux pages web
- ✓ Variables : let, const — types : string, number, boolean
- ✓ Fonctions : declaration avec function, return
- ✓ Conditions : if / else
- ✓ DOM : document.getElementById(), textContent, style
- ✓ Événements : onclick, addEventListener

Exercice réalisé

Créer un bouton interactif qui modifie le contenu de la page, puis une mini calculatrice avec addition et multiplication utilisant deux champs de saisie.

Code essentiel

```
function changerMessage() {  
  let msg = document.getElementById("messageSpecial");  
  msg.textContent = "Bravo Jopie !";  
  msg.style.color = "#2980b9";  
}  
  
function additionner() {  
  let n1 = Number(document.getElementById("nombre1").value);  
  let n2 = Number(document.getElementById("nombre2").value);  
  document.getElementById("resultat").textContent = n1 + n2;  
}
```

■ **Accomplissement :** Mini calculatrice fonctionnelle — $15 \times 7 = 105$ ✓

Semaine 5 ■ DOM & To-Do List

Objectif : Maîtriser la manipulation avancée du DOM JavaScript.

Concepts clés appris

- ✓ DOM = Document Object Model : représentation de la page
- ✓ Tableaux (arrays) et méthode push(), splice(), filter()
- ✓ Boucle forEach() pour parcourir les éléments
- ✓ createElement() et appendChild() pour créer des éléments
- ✓ classList.add() et classList.remove() pour les classes CSS
- ✓ Template literals avec les backticks `` pour l'HTML dynamique

Exercice réalisé

Créer une To-Do List complète avec : ajout de tâches, marquage comme terminé (biffé en vert), suppression, compteur de tâches terminées et touche Entrée pour ajouter.

Code essentiel

```
let taches = [];  
  
function ajouterTache() {  
  let texte = document.getElementById("inputTache").value;  
  taches.push({ texte: texte, termine: false });  
  afficherTaches();  
}  
  
function toggleTermine(index) {  
  taches[index].termine = !taches[index].termine;  
  afficherTaches();  
}
```

■ **Accomplissement : Application To-Do List complète et interactive**

Semaine 6 ■ Git & GitHub

Objectif : Versionner le code et le publier sur Internet avec Git et GitHub.

Concepts clés appris

✓ Git = machine à remonter le temps pour le code

✓ Repository (dépôt) : dossier surveillé par Git

✓ Commit : photo du code à un instant précis

✓ Branch : branche de développement (main = principale)

✓ GitHub : hébergement du code en ligne

✓ GitHub Pages : publier un site web gratuitement

Exercice réalisé

Installer Git, configurer identité, créer un repository local avec git init, faire le premier commit, créer un repo GitHub et pousser le code avec git push.

Code essentiel

```
git config --global user.name "Jopie"
git config --global user.email "nlemvojopie@gmail.com"

git init
git add .
git commit -m "Premier commit - Ma carte de visite"
git branch -M main
git remote add origin https://github.com/...
git push -u origin main
```

■ **Accomplissement : Site web en ligne : nlemvojopie-star.github.io/formation-web**

Semaine 7 ■ Prompt Engineering

Objectif : Maîtriser l'art de communiquer avec l'IA pour obtenir des résultats excellents.

Concepts clés appris

✓ Prompt = message envoyé à l'IA (Claude, GPT...)

✓ Technique 1 : Définir un rôle ('Tu es un expert en...')

✓ Technique 2 : Être précis et détaillé dans sa demande

✓ Technique 3 : Demander un format précis (tableau, JSON...)

✓ Technique 4 : Chain-of-thought (raisonnement étape par étape)

✓ Technique 5 : Itérer et affiner progressivement

Exercice réalisé

Tester 3 niveaux de prompts sur Claude, faire analyser et corriger du code cassé, utiliser Claude pour améliorer le portfolio avec des conseils pro (structure sémantique, accessibilité, variables CSS).

Code essentiel

```
# Formule du prompt parfait :  
"Tu es [ROLE].  
Ta tâche : [TACHE PRECISE].  
Contexte : [INFORMATIONS UTILES].  
Format souhaité : [FORMAT].  
Contraintes : [LIMITES]."
```

■ **Accomplissement : Portfolio redesigné avec aide IA — avatar JN, header pro, carte centrée**

Semaine 8 ■ Python

Objectif : Découvrir Python, le langage de l'IA et de l'automatisation.

Concepts clés appris

- ✓ Python = langage numéro 1 mondial pour l'IA
- ✓ `print()` : afficher du texte dans le terminal
- ✓ Variables : `prenom = 'Jopie', age = 50, est_developpeur = True`
- ✓ Listes : `competences = ['HTML', 'CSS', 'JavaScript']`
- ✓ Boucle `for` : parcourir une liste élément par élément
- ✓ Fonctions : `def ma_fonction(parametre): ... return resultat`

Exercice réalisé

Créer `premier_pas.py` avec variables et opérations, puis `fonctions.py` avec fonctions personnalisées incluant calcul d'âge et évaluation de niveau selon nombre de compétences.

Code essentiel

```
def dire_bonjour(prenom):  
    return "Bonjour " + prenom + " !"   
  
def evaluer_niveau(nb_competences):  
    if nb_competences >= 5:  
        return "Niveau Expert !"   
    elif nb_competences >= 3:  
        return "Niveau Intermediaire"   
    else:  
        return "Niveau Debutant"   
  
niveau = evaluer_niveau(5)  
print("Ton niveau :", niveau)
```

■ **Accomplissement : Programmes Python fonctionnels — Niveau Expert confirmé !**

Tableau des Codes Essentiels

HTML — Structure

Balise	Rôle	Exemple
<!DOCTYPE html>	Déclare le type de document	Toujours en 1ère ligne
<html lang='fr'>	Racine de la page	Contient tout le code
<head>	Informations invisibles	Titre, CSS, meta
<body>	Contenu visible	Tout ce que l'user voit
<h1> à <h6>	Titres et sous-titres	Mon titre
<p>	Paragraphe de texte	Bonjour Jopie !
+	Liste à puces	Item
	Lien cliquable	Texte
	Image	
<div>	Conteneur générique	<div class='carte'>...</div>
<header>	En-tête sémantique	Haut de page
<main>	Contenu principal	Corps de page
<footer>	Pied de page sémantique	Bas de page
<button>	Bouton cliquable	<button onclick='fn()'>
<input>	Champ de saisie	<input type='text' id=''>

CSS — Apparence

Propriété	Rôle	Exemple
<code>background-color</code>	Couleur de fond	<code>background-color: #f0f4f8</code>
<code>color</code>	Couleur du texte	<code>color: #333333</code>
<code>font-family</code>	Police d'écriture	<code>font-family: 'Poppins'</code>
<code>font-size</code>	Taille du texte	<code>font-size: 1.2rem</code>
<code>font-weight</code>	Épaisseur du texte	<code>font-weight: bold</code>
<code>text-align</code>	Alignement du texte	<code>text-align: center</code>
<code>padding</code>	Espace intérieur	<code>padding: 20px</code>
<code>margin</code>	Espace extérieur	<code>margin: 0 auto</code>
<code>border</code>	Bordure	<code>border: 2px solid blue</code>
<code>border-radius</code>	Coins arrondis	<code>border-radius: 12px</code>
<code>box-shadow</code>	Ombre portée	<code>box-shadow: 0 4px 20px rgba(0,0,0,.1)</code>
<code>display: flex</code>	Disposition flexible	<code>display: flex</code>
<code>justify-content</code>	Alignement horizontal	<code>justify-content: center</code>
<code>transition</code>	Animation douce	<code>transition: all 0.3s</code>
<code>:hover</code>	Effet au survol	<code>a:hover { color: red }</code>
<code>@import</code>	Importer une police	<code>@import url('fonts.googleapis...')</code>
<code>max-width</code>	Largeur maximale	<code>max-width: 700px</code>

JavaScript — Comportement

Code JS	Rôle	Exemple
<code>let / const</code>	Déclarer une variable	<code>let prenom = 'Jopie'</code>
<code>console.log()</code>	Afficher dans la console	<code>console.log('Bonjour')</code>
<code>alert()</code>	Afficher une popup	<code>alert('Message !')</code>
<code>function</code>	Créer une fonction	<code>function direBonjour() {}</code>
<code>return</code>	Retourner une valeur	<code>return resultat</code>
<code>if / else</code>	Condition	<code>if (age >= 18) { ... }</code>
<code>for</code>	Boucle	<code>for (let i=0; i<5; i++) {}</code>
<code>forEach()</code>	Boucler sur un tableau	<code>liste.forEach(item => {})</code>
<code>getElementById()</code>	Trouver un élément HTML	<code>document.getElementById('id')</code>
<code>.textContent</code>	Modifier le texte	<code>element.textContent = 'Texte'</code>
<code>.style.color</code>	Modifier le style	<code>element.style.color = 'red'</code>
<code>.innerHTML</code>	Modifier le HTML	<code>element.innerHTML = 'Gras'</code>
<code>addEventListener</code>	Écouter un événement	<code>el.addEventListener('click', fn)</code>
<code>onclick</code>	Clic sur un élément	<code><button onclick='maFonction()></code>
<code>.push()</code>	Ajouter à un tableau	<code>tableau.push(nouvelElement)</code>
<code>.splice()</code>	Supprimer du tableau	<code>tableau.splice(index, 1)</code>
<code>Number()</code>	Convertir en nombre	<code>Number('42') → 42</code>
<code>Template literal</code>	Texte avec variables	<code>`Bonjour \${prenom} !`</code>

Git — Versioning

Commande	Rôle
<code>git init</code>	Initialiser un dépôt Git dans le dossier
<code>git add .</code>	Préparer tous les fichiers pour le commit
<code>git commit -m 'msg'</code>	Sauvegarder une version du code
<code>git branch -M main</code>	Renommer la branche principale
<code>git remote add origin URL</code>	Lier au dépôt GitHub distant
<code>git push -u origin main</code>	Envoyer le code sur GitHub
<code>git push</code>	Mettre à jour GitHub après modifications
<code>git status</code>	Voir l'état des fichiers
<code>git log</code>	Voir l'historique des commits
<code>cd [dossier]</code>	Naviguer dans un dossier
<code>python fichier.py</code>	Exécuter un script Python

Python — Le langage de l'IA

Code Python	Rôle	Exemple
<code>print()</code>	Afficher du texte	<code>print('Bonjour Jopie !')</code>
<code>variable = valeur</code>	Stocker une donnée	<code>prenom = 'Jopie'</code>
<code>str / int / bool</code>	Types de données	<code>'texte' / 42 / True</code>
<code>liste = []</code>	Créer une liste	<code>competences = ['HTML', 'CSS']</code>
<code>len(liste)</code>	Compter les éléments	<code>len(competences) → 5</code>
<code>for x in liste:</code>	Boucler sur une liste	<code>for c in competences:</code>
<code>if / elif / else:</code>	Conditions	<code>if age >= 18: ...</code>
<code>def fonction():</code>	Définir une fonction	<code>def dire_bonjour(prenom):</code>
<code>return valeur</code>	Retourner un résultat	<code>return 'Bonjour ' + prenom</code>
<code># commentaire</code>	Commenter le code	<code># Ceci est un commentaire</code>

Ma Procédure en 10 Étapes

1	PENSER Réfléchis avant de coder : Quelle est ma page ? Quel contenu ? Quel design ? Quelles actions ?
2	SCHÉMA Dessine sur papier la structure avec des rectangles : Header, Contenu, Footer.
3	HTML D'ABORD Construis la structure avant l'apparence ! Crée index.html, tape ! + Tab, ajoute le contenu.
4	CSS ENSUITE Stylise une propriété à la fois, teste à chaque Ctrl+S. Ordre : body → header → contenu → footer.
5	JAVASCRIPT Ajoute les interactions seulement si nécessaire. Relie script.js avant </body>.
6	IA COMME OUTIL Décris ton problème à Claude précisément. Analyse le code généré, ne jamais copier sans comprendre.
7	DÉBOGUER F12 → Console → Lis l'erreur. Cherche sur Google en anglais. Vérifie majuscules et points-virgules.
8	GIT RÉGULIER Après chaque avancée : git add . → git commit -m 'description' → git push.
9	TESTER PARTOUT Vérifie sur Chrome, Firefox et en mode mobile (F12 → icône mobile) avant de publier.
10	PUBLIER git push → GitHub Pages génère le site → Partage ton lien nlemvojopie-star.github.io

Règle d'or : PENSER → PRATIQUER → PUBLIER

Mes Accomplissements

En 8 semaines, je suis passé de zéro à développeur web opérationnel !

Semaine	Accomplissement	Statut
1	Environnement VSCode + Node.js + Terminal opérationnel	✓
2	Première page web HTML créée et visible dans le navigateur	✓
3	Page stylisée avec CSS, Google Fonts Poppins et animations	✓
4	Mini calculatrice JavaScript fonctionnelle (15 x 7 = 105)	✓
5	To-Do List interactive complète avec compteur de tâches	✓
6	Code publié sur GitHub + Site en ligne sur GitHub Pages	✓
7	Portfolio redesigné avec aide IA — avatar JN, design pro	✓
8	Premiers programmes Python — Niveau Expert confirmé !	✓

Compétences acquises

HTML	CSS	JavaScript	Python	Git	GitHub	Prompt IA
Expert	Avancé	Intermédiaire	Débutant+	Maîtrisé	Maîtrisé	Expert

Mon site en ligne :
nlemvojopie-star.github.io/formation-web

Mon GitHub :
github.com/nlemvojopie-star

"La rigueur, la curiosité et la persévérance
sont les clés du développeur de demain."
— Jopie Nlemvo, Mai 2026